AIORI 3 HACKATHON

# Incident Timeline and Evidence Collection for Small Teams

Problem Statement 13 (PS-013)

| Field | Details |
|---|---|
| Project focus | A lightweight, evidence-first workflow for small incident-response teams. |
| Prototype | Evidence manifest with SHA-256, normalized timeline, evidence relationships, and human-reviewed report. |
| Environment | Controlled Windows 11 virtual machine and Kali/Ubuntu analysis environment; benign test activity only. |
| Team / institution | To be completed by the team before submission. |
| Date | October 2026; confirm the official submission date. |

Prepared using the supplied 11-page structure. Verify team names, official team ID, and event metadata before submission.

# Abstract

Small incident-response teams often investigate incidents using evidence scattered across endpoint logs, file metadata, system events, and analyst notes. Manually collecting artifacts, checking integrity, converting timestamps, and reconstructing an event sequence can consume valuable time and create gaps. This project proposes a practical prototype for incident timeline generation and evidence collection. It records provenance and SHA-256 hashes, normalizes event fields and timestamps, correlates related events, and presents a searchable timeline with supporting evidence. An optional AI assistant may summarize selected structured events, but it must cite evidence IDs and never replace deterministic checks or analyst judgment. The prototype will be evaluated using controlled, benign scenarios and documented ground truth. Metrics include processing time, event coverage, hash verification, and report usefulness. It is intended as a low-complexity workflow, not a replacement for enterprise SIEM or forensic platforms.

# Problem Description

PS-013 addresses the difficulty of collecting digital evidence and reconstructing a reliable incident timeline when a team has limited staff, budget, and specialist tooling.

## Key issues

• Evidence is fragmented across logs, endpoint artifacts, file metadata, and analyst notes.

• Sources differ in timestamp formats, time zones, field names, and detail.

• Manual transformation can lose provenance or introduce errors.

• A chronological list alone does not explain event relationships or why an event matters.

• Small teams need consistent, reviewable investigation reports.

• AI summaries may be incomplete or incorrect unless tied to verifiable evidence.

## Objective

Build a prototype that preserves originals, records provenance and hashes, normalizes and correlates events, displays a traceable timeline, and produces a human-reviewed summary. Operate only on authorized systems and data.

# Background

Digital forensics and incident response (DFIR) relies on trustworthy observations from multiple sources. A timeline combines timestamped artifacts so investigators can reason about what happened, when it happened, and which evidence supports a conclusion. Plaso provides a super-timeline approach for correlating information from logs and other files [3]. NIST SP 800-61 Rev. 3 integrates incident response into cybersecurity risk management [1].

## Core concepts

• **Evidence provenance:** record source, collection time, collector, and tool.

• **Integrity:** calculate SHA-256 and verify transferred copies. A matching hash does not by itself prove authenticity or completeness.

• **Normalization:** map records to a common schema while retaining original fields.

• **Timeline correlation:** order events and connect shared host, user, process, file, or network attributes where available.

• **Human review:** separate observed facts from hypotheses; analysts decide whether evidence supports a conclusion.

## Scope and assumptions

Use a controlled Windows 11 virtual machine and benign documented actions. Do not execute ransomware, exploit systems, collect credentials, or monitor third-party devices. The prototype is not a claim of legal forensic certification or enterprise-scale performance.

## Recent research context

A 2025 study proposes ground-truth-based evaluation for LLM timeline tasks, supporting controlled and documented testing rather than anecdotal demonstrations [4].

# Literature & RFC Review

Selected sources inform incident response, timeline generation, and the safe use of AI in digital forensic analysis.

| Source | Contribution | Relevance / limitation |
|---|---|---|
| NIST SP 800-61 Rev. 3 (2025) [1] | Incident-response guidance aligned with NIST CSF 2.0. | Provides process context; it is not a ready-made timeline application. |
| Loumachi et al., GenDFIR (2025) [2] | Explores RAG and LLM support for cyber incident timeline analysis. | Informs optional AI summaries; outputs still need evidence-based validation. |
| Studiawan et al. (2025) [4] | Proposes standardized evaluation, dataset, and ground truth for LLM timeline analysis. | Supports repeatable testing; not a complete small-team evidence workflow. |
| Plaso documentation [3] | Python-based engine for super timelines from artifacts and logs. | Timeline extraction is one component, not the full case/evidence workflow. |
| Velociraptor documentation [5] | Endpoint collection and investigation using artifacts and VQL. | Potential future integration; may be complex for the first MVP. |

## Research gap and positioning

Do not claim that timeline automation is novel. The intended contribution is an accessible integrated workflow: case-linked evidence manifest, hash verification, transparent correlation, timeline with evidence links, and human-reviewed report. AI remains an optional assistant, not the source of truth.

## Evaluation principle

Compare generated events with documented lab ground truth. No performance results are claimed until implementation and testing are complete.

# Proposed Solution

The solution is a modular evidence-to-timeline pipeline. The MVP imports authorized exported logs and selected files; later versions may add endpoint collection integration. Acquisition is separated from analysis to protect original evidence.

| Stage | Function | Output |
|---|---|---|
| 1. Case intake | Case ID, scope, investigator, time window. | Case record |
| 2. Evidence acquisition | Import logs/files and record provenance. | Evidence manifest |
| 3. Integrity check | Calculate SHA-256 and verify transfers. | Hash status |
| 4. Parse / normalize | Map supported records to a common event schema. | Normalized events |
| 5. Correlate | Sort events and connect shared identifiers. | Timeline and relationships |
| 6. Explain / review | Show evidence-linked findings; optional bounded summary. | Reviewed findings |
| 7. Report | Chronology, manifest, hashes, assumptions. | CSV/JSON/PDF report |

## Proposed technology stack

| Component | Purpose |
|---|---|
| Python + Pandas | Parsing, normalization, time conversion, tabular event handling. |
| SQLite | Store case metadata, evidence manifest, and events. |
| SHA-256 / hashlib | Integrity verification of files and transferred copies. |
| Plotly + Streamlit | Interactive timeline, filters, overview, report preview. |
| NetworkX (optional) | Evidence relationship view; every edge should be traceable. |
| Plaso (optional) | Extract a broader timeline from supported forensic artifacts. |
| Velociraptor (optional) | Repeatable authorized endpoint collection if time permits. |
| LLM/RAG (optional, last) | Summarize selected events with evidence IDs; analyst approval required. |

# Solution Optimization

Optimization reduces avoidable manual effort without weakening evidence quality. Originals are retained so an analyst can audit transformations.

| Area | Before | After | Verification |
|---|---|---|---|
| Evidence tracking | Inconsistent folder naming. | Case-linked manifest and SHA-256. | Recompute hashes. |
| Timestamps | Manual comparison. | Normalize to UTC; retain original values. | Test known timestamps. |
| Timeline | Copy/paste into sheets. | Parser creates a common schema. | Compare with ground truth. |
| Correlation | Visual search for repeated IDs. | Transparent rules and evidence links. | Record rule and source IDs. |
| Reporting | Written from memory. | Draft from reviewed timeline and manifest. | Human fact-check. |
| AI | Free-form summary. | Bounded summary with evidence IDs. | Check unsupported claims. |

## Optimization rules

• Keep originals separate from derived outputs.

• Filter to the relevant time window before optional AI summarization.

• Do not send full raw evidence collections to an LLM.

• Flag missing timestamps, duplicates, parse failures, and hash mismatches explicitly.

• Use deterministic code for hashing, sorting, and rule-based links; AI may assist with language only.

• Treat a shared IP or filename as a possible relationship, not proof of maliciousness.

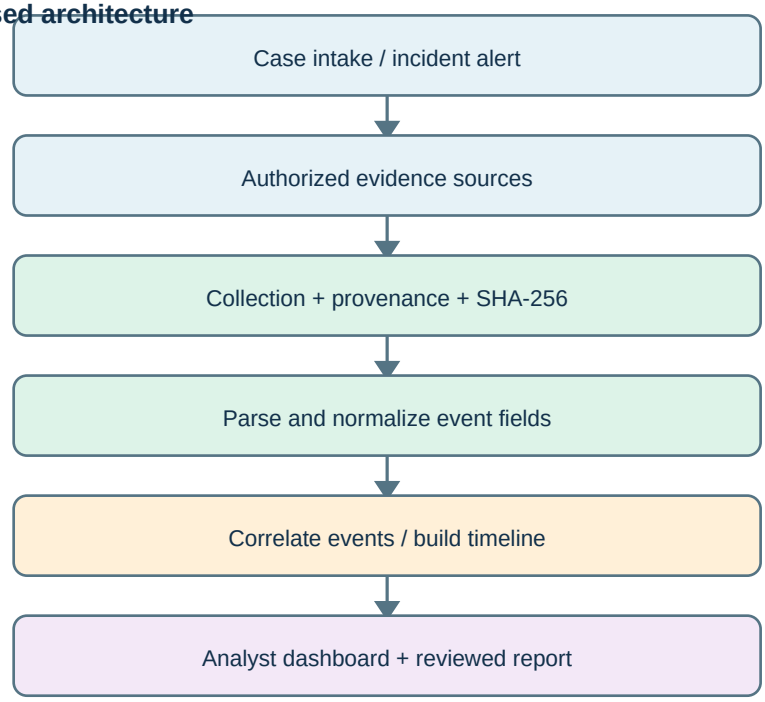
## Expected benefit

Consistency, traceability, and review speed are expected benefits to be measured—not results already achieved. Actual measurements and failures will be reported after testing.

# Architecture Design (UML / Component View)

The architecture separates evidence acquisition, integrity controls, event processing, and analyst-facing outputs. Optional integrations are added after the core pipeline works.

**Figure 1. Proposed architecture**




*Original evidence remains separate; AI assistance is optional and human-reviewed.*

## Component responsibilities

- **Case intake:** defines authorized scope and case context.
- **Evidence importer:** accepts exported logs and records source metadata.
- **Integrity service:** calculates and verifies SHA-256.
- **Normalizer:** retains source IDs and original timestamps.
- **Correlation engine:** explains each link with supporting event IDs.
- **Dashboard/report:** presents filters, chronology, evidence, and limitations.
- **Optional AI assistant:** summarizes selected structured events for human verification.

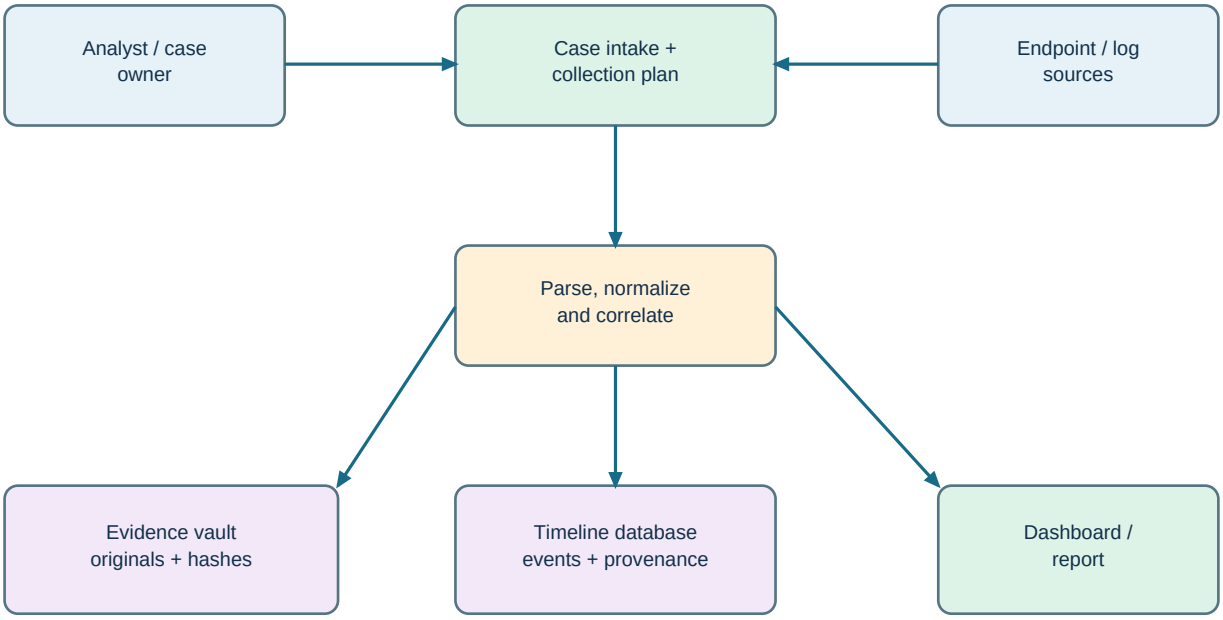
## Design decision

The MVP does not require a live enterprise agent or a large language model. A folder of authorized exported logs is enough to demonstrate the pipeline, reducing installation risk and allowing integrity and timeline logic to be validated first.

# Data Flow Diagram (DFD)

The diagram shows how case details, endpoint records, original evidence, normalized events, and final findings move through the system. Original evidence is separated from derived records.

**Figure 2. Logical data-flow diagram**




*Use only lab data or systems for which permission is documented.*

## Data handling and trust boundaries

| Flow / store | Contents | Control |
|---|---|---|
| Case intake | Case ID, authorization, time window, notes. | Document scope before collection. |
| Endpoint/log sources | Exported EVTX, CSV, JSON, selected artifacts. | Use lab-generated or authorized data only. |
| Evidence vault | Original files, source metadata, SHA-256. | Keep originals separate; verify copies. |
| Timeline store | Normalized events, source IDs, parse status, normalized time. | Retain original timestamps and errors. |
| Report | Chronology, correlations, evidence links, caveats. | Human review before finalization. |

## Security and privacy controls

- Use synthetic or lab-created activity wherever possible; redact personal data before sharing.
- Keep evidence separate from generated reports and temporary files.
- Do not overwrite originals during parsing or normalization.
- Record software versions, time-zone assumptions, and collection steps.
- Mark unsupported formats and failed parses clearly; do not silently omit them.

# Deployment Timeline

Milestones are relative to project start and should be adapted to the actual hackathon schedule.

| Milestone | Work items | Exit criteria |
|---|---|---|
| M1 Scope / environment | Confirm requirements, repository, Windows VM and analysis VM. | Written scope and environment checklist. |
| M2 Controlled evidence | Generate benign actions, export logs, record expected sequence. | Evidence folder and ground truth. |
| M3 Integrity manifest | Inventory files, capture metadata, hash and re-verify. | Complete manifest. |
| M4 Parser / schema | Parse CSV/JSON first; add EVTX support if feasible. | Tests pass; failures flagged. |
| M5 Timeline / correlation | Sort events and add transparent correlation rules. | Timeline checked against ground truth. |
| M6 Dashboard / report | Build filters, timeline view, evidence details, export. | End-to-end demonstration. |
| M7 Optional integrations | Assess Plaso/Velociraptor; add AI last if justified. | Only stable integrations retained. |
| M8 Evaluation / demo | Repeat trials and measure quality and timing. | Results, limitations, reproducible demo. |

## Test methodology

• Create a benign scenario with documented actions and expected events.

• Repeat the scenario at least three times where practical; retain runs separately.

• Compare event coverage, missed events, duplicates, and order against ground truth.

• Measure import, hashing, parsing, timeline, and report time.

• Record parse errors, time-zone assumptions, and analyst corrections.

## Evaluation metrics

Hash verification; event coverage; missed and duplicate events; timestamp/order accuracy; processing time; unsupported AI claims; and analyst rating of report clarity. No numerical result is claimed before testing.

# Bibliography

References consulted for the proposed design. Official documentation and peer-reviewed research are used where available; project-specific performance claims remain unverified until testing.

[1] A. Nelson, S. Rekhi, K. Scarfone, and M. Souppaya, *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*, NIST SP 800-61 Rev. 3, Apr. 2025. https://csrc.nist.gov/pubs/sp/800/61/r3/final

[2] F. Y. Loumachi, M. C. Ghanem, and M. A. Ferrag, “Advancing Cyber Incident Timeline Analysis Through Retrieval-Augmented Generation and Large Language Models,” *Computers*, vol. 14, no. 2, article 67, 2025. doi: 10.3390/computers14020067. https://www.mdpi.com/2073-431X/14/2/67

[3] Plaso Project, “Welcome to the Plaso documentation,” Plaso (log2timeline) documentation. https://plaso.readthedocs.io/en/latest/

[4] H. Studiawan, F. Breitinger, and M. Scanlon, “Towards a standardized methodology and dataset for evaluating LLM-based digital forensic timeline analysis,” *Forensic Science International: Digital Investigation*, vol. 54, supplement, article 301982, Oct. 2025. doi: 10.1016/j.fsidi.2025.301982. https://www.sciencedirect.com/science/article/pii/S2666281725001222

[5] Velociraptor Project, “Velociraptor documentation,” endpoint collection and digital forensic investigation documentation. https://docs.velociraptor.app/

## Notes on evidence and claims

- Literature supports timeline reconstruction, structured evidence processing, and ground-truth-based evaluation.
- The proposed workflow is a design proposal, not a claim that existing tools lack these capabilities.
- Verify current installation steps against official documentation during implementation.
- Add team names, institution, official team ID, and acknowledgements before submission.

Project status: proposed design; implementation and benchmark results pending.

# THANK YOU

Incident Timeline and Evidence Collection for Small Teams

PS-013 | AIORI 3 Hackathon

Evidence first. Traceable timelines. Human-reviewed conclusions.

## Questions and feedback

The team welcomes feedback on scope, evidence integrity controls, timeline accuracy, usability for small teams, and evaluation methodology.

Reminder: demonstrations must use controlled lab data or systems for which authorization is documented.